

FEBRUARY 21, 2026

PROJECT PLAN

TEAM: 4 - INTELLIGENT BICYCLE DASH CAM

PREPARED BY: TEAM 4

MATTHEW EDLIN

COLIN TUGEND

JAIR FRANCO

CHRISTOPHER PAQUIN

TABLE OF CONTENTS

Project Description.....	2
Overview (CO-5):.....	2
Key Requirements (CO-5):.....	2
Deliverables (CO-5, CO-6):	2
Acronyms and Abbreviations (CO-7):	3
Design and Architecture (CO-1, CO-3)	3
Implementation Strategy	7
High-level Work Breakdown Structure (CO-2):	7
Schedule / Timeline (CO-2):	7
Required Hardware (CO-2):.....	12
Third party content (CO-2):.....	14
Quality (CO-2):	14
Other Special considerations (CO-7, CO-3):	16
Process	17
Process Description and Justification (CO-2)	17
Tools (CO-2):.....	17
Roles and Responsibilities (CO-2):.....	17
Location of Project Artifacts (CO-2):	17
Sponsor Communications (CO-7):.....	18
Risk management.....	18
Identified Potential Risks (CO-2):	18
Mitigation Strategies (CO-2, CO-3):.....	18

PROJECT DESCRIPTION

OVERVIEW:

The Intelligent Bicycle Dash Cam (IBDC) project helps cyclists document unsafe vehicle passing. The handlebar device detects when a vehicle passes within 3 feet and records images of the license plate. This project will build a smartphone app for Android and iOS that connects to the device, pulls incident data, and helps the rider review, and export incidents for reporting. The primary users will be cyclists using an IBDC device. Secondary users include law enforcement and the Rob Dollar foundation.

GLOBAL TRENDS:

Global trends show growing demand for cyclists' safety. Cities are increasingly adding bike lanes and other safety programs due to more people using bikes for commuting and exercise. At the same time, drivers are more distracted than ever due to expanding communication systems, and bike/car incidents are a common safety concern. Because smartphones are so prevalent, they make an excellent platform to use to report incidents and support safer road policies.

MARKET ANALYSIS:

Alternatives are often problematic with the best alternative being a GoPro camera. Many options require the user to record an entire run using lots of battery power and digital storage or require the user to perform a task like taking a picture with their phone which is challenging when they are worried about being hit by a car. This product fills a genuine void in the market with a clear need.

SECURITY CONSIDERATIONS:

There are no security concerns with this application currently. Incident data will be images and user added data along with GPS and time. None of this is private or at risk of exposing the user to monetary or property loss. No account system or log-in system will be required. In summary, at this point there is no need to encrypt or protect the user's data.

KEY REQUIREMENTS:

Deliverables include implementing a device-to-smartphone interface to exchange settings, retrieving incident data such as distance and images, and pull device status information such as battery level and SD card usage, using the provided interface specification. The app will capture the time and GPS coordinates for each incident when it is reported by the device. It will also allow users to add additional information to incidents, including general comments, injury severity, whether the driver left or stayed, driver identity details, license plate number, and vehicle make/model/color. Incidents and all associated data will be stored on the smartphone, with support for viewing, editing, and deleting incidents, including removing images in the case of false flags. The application will provide image browsing for incidents and allow the user to select the best image. A dedicated Device page will support Bluetooth pairing, display connected device status, and allow configuration of device settings. This page will be extensible for future settings. A Settings page will allow users to manage app-level preferences and will be designed to accommodate additional settings over time.

DELIVERABLES:

Deliverables for the project include the core mobile application that will function on Android and iOS. The UI module will provide all required screens, including a main menu, view and edit incidents screen, file report screens, and settings screen. The Incident Handling module will support receiving, storing, editing, and saving incident data, including images, location, notes, and other metadata. The Connectivity module will include Data Access Objects for database access and Protobuf handling for the sensor/camera interface. Finally, the App Settings module will implement approved user customizations such as images per capture and supported file export formats.

ACRONYMS AND ABBREVIATIONS:

PDF	Portable Document Format
OCR	Optical Character Recognition
IBDC	Intelligent Bicycle Dash Cam
UI	User Interface
IOS	iPhone Operating System
UML	Unified Modeling Language
CI/CD	Continuous Integration / Continuous Delivery
SOLID	Design Principles
DAO	Database Access Object
SDK	Software Development Kit
CLI	Command Line Interface
IDE	Integrated Development Environment
MVVM	Model-ViewModel-View

DESIGN AND ARCHITECTURE:**Guiding Principles of Design:**

The IDBC system can be viewed as a collection of related components, where each component encapsulates related data and functionality. Viewing the system in terms of related components has the benefits of modularity and replaceability. A modular system is a system where subsystems (modules) work together to achieve a set of functionalities. Each module contributes in some way to the overall solution. Modularity is favorable because it lends a system to be extensible and decoupled. A replaceable system is a system where units of functionality can be swapped for other units as long as they provide the same functionality. Replaceability is favorable because it allows different new implementations of a solution to easily replace existing components.

Decisions made throughout the design and architecture of the IDBC system will exemplar the concepts of modularity and replaceability. Below is a high-level Use Case diagram outlining the

different actors that interact with the system. The different actors can be viewed as a module or something required to facilitate the behavior of a module (see fig 1).

In Fig 1 the stick figures (Actors) can be viewed as modules within the system that play a defined role in the application. Each oval (Use Case) attached to the actors is a way in which the actor interacts with the system. Every actor can be mapped into a component, and each use case can be mapped into a requirement. This will serve as the basis for a more detailed SRS document.

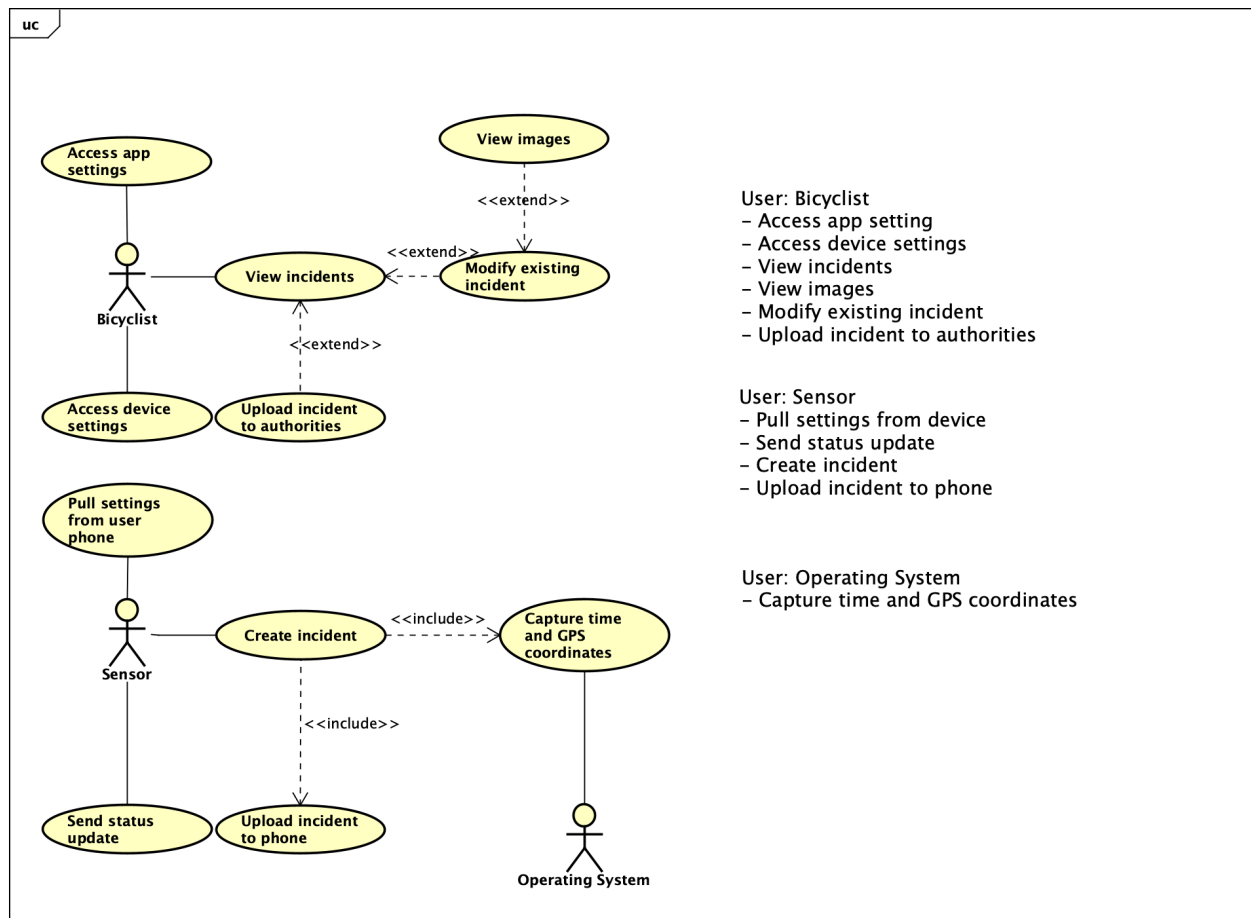


Fig 1. Use Case Diagram modeling the system behaviors and their agents

Although components are separate entities, sometimes they need to communicate with each other to achieve the required functionality. Fig 2 demonstrates how two components work together to create an incident.

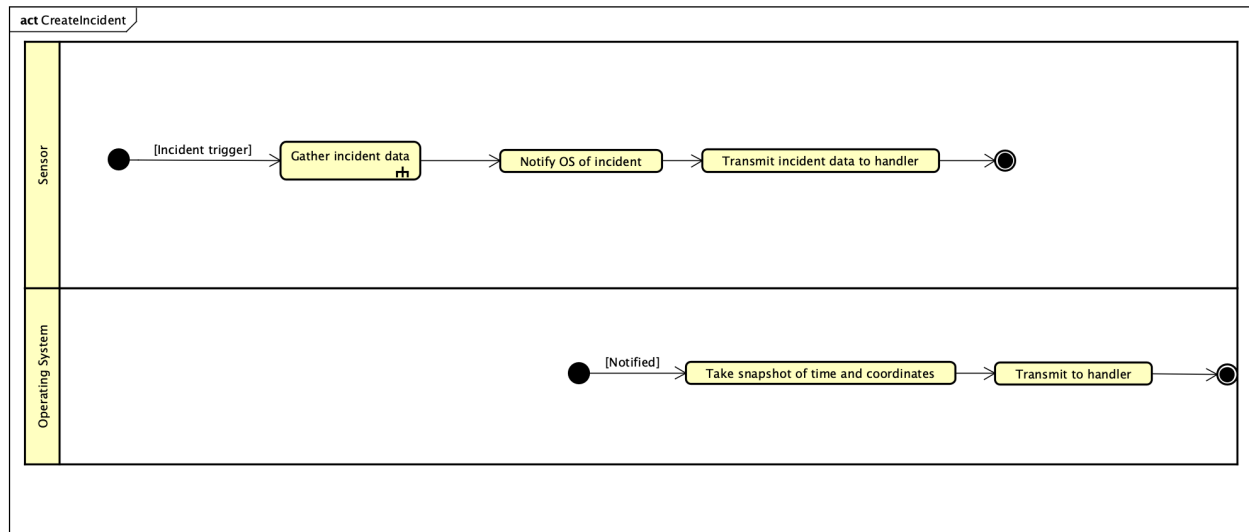


Fig. 2 Demonstrates communication between modules

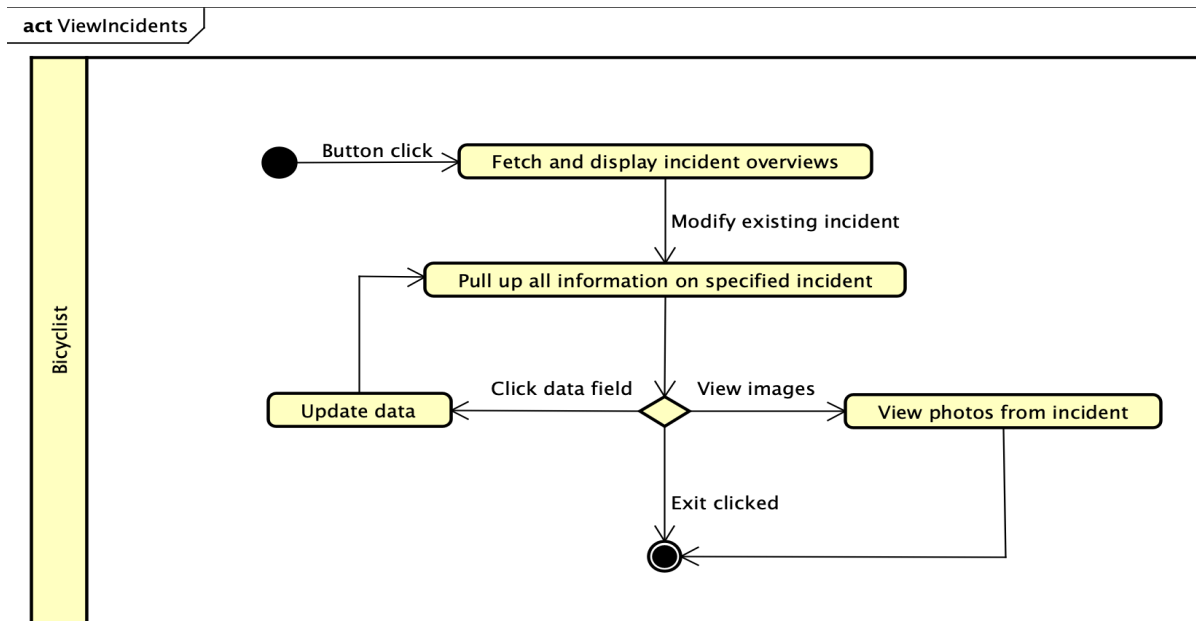
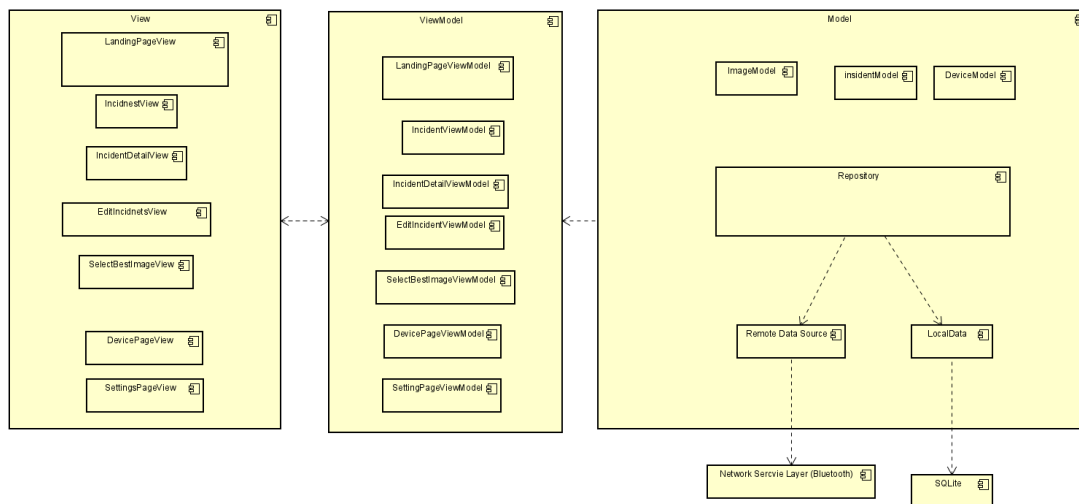


Fig. 3 Demonstrates communication within a module

DESIGN DESCRIPTION (ABET-1, ABET-2)

This design uses MVVM architecture. The purpose is to keep the UI separate from the business logic and data layer. This is great for mobile applications. The View contains all the components that are associated with the UI. For this application, they are separated based on use cases derived from the project. These sub-components are LandingPageView, IncidentView, IncidentDetailView, EditIncidentView, SelectBestImageView, DevicePageView, and SettingsPageView. While the View handles the UI components, the ViewModel contains the business logic to handle events and updates the views accordingly by calling the model and updating the corresponding ViewModel. The ViewModel components are updated by the Model. The model is the data layer in charge of loading and saving data (locally and from Bluetooth) and packaging that data for the ViewModel.



ALTERNATE DESIGN POSSIBILITIES (EM@FSE-B)

The design could be implemented using layered architecture. This will keep the system simple and implementable within the given timeframe. The components of this layered architecture will consist of a presentation layer, business layer, and technical layer. The presentation layer will consist of the UI elements such as the views, Ui-states, navigation, and input validations. The business layer will orchestrate workflows and consist of elements that relate to uses cases such as “retrieving incidents”, “update information”, and “Location capturing.” The technical layer will talk to the hardware interface and be responsible for storing and retrieving data.

IMPLEMENTATION STRATEGY

HIGH-LEVEL WORK BREAKDOWN STRUCTURE:

Our high-level work breakdown takes the 5 primary modules and breaks them down into more manageable tasks that can be used to map our progress and plan a schedule. There are also development requirements like testing and documentation that have been added.

APP CORE

Task 1.1	Project Scaffolding
Scope:	Setup project in GitHub. Setup Expo. Implement static analysis.
Skillset	Software engineering principles, CI/CD, Version control
Estimated Time:	4 hours

Task 1.2	Application architecture structure
Scope:	Define responsibilities of Model, View, View controller and data access layer.
Skillset	Software engineering principles, Mobile development, SOLID
Estimated Time:	3 hours

Task 1.3	Application permissions
Scope:	Request and handle permissions for GPS. Likely specific to operating system.
Skillset	Mobile development
Estimated Time:	4 hours

Task 1.4	Setup embedded database
Scope:	Add embedded database, determine schema
Skillset	Database Management, Mobile Development
Estimated Time:	4 hours

Task 1.5	App Core Testing
Scope:	Build & run application. Load mock data into database. Test CI/CD & repo cloning
Skillset	Database Management, Mobile Development, Version control
Estimated Time:	5 hours

UI MODULE

Task 2.1	UI Scaffolding
Scope:	Create a simple and distinguishable screen/view for each screen required
Skillset	UI development, Typescript/React
Estimated Time:	4 hours

Task 2.2	Update screens to Minimum viable product quality
Scope:	Add core interactable and display functionality
Skillset	UI development, Typescript/React
Estimated Time:	3 hours per screen

Task 2.3	UI Navigation
Scope:	Establish UI navigation between screens.
Skillset	UI development, Typescript/React
Estimated Time:	6 hours

Task 2.4	UI Documentation
Scope:	Document the intent of each screen and what needs to be added or changed.
Skillset	UI Development, Software Engineering principles
Estimated Time:	6 hours

Task 2.5	UI Testing
Scope:	Test screen navigation and redraw. Verify system status bars. Attempt to crash the app.
Skillset	UI Testing, Software Engineering principles
Estimated Time:	3 hours

INCIDENT HANDLING

Task 3.1	Data Interface 1
Scope:	Create class(s) to get data from device and store it in the database.
Skillset	Software Engineering, Typescript
Estimated Time:	4 hours

Task 3.2	Data Interface 2
Scope:	Create class(s) to pull incidents from database and allow editing.
Skillset	Software Engineering, Typescript
Estimated Time:	4 hours

Task 3.3	Service modules
Scope:	Create class(s) to capture GPS location and time from device. (IOS/Android specific)
Skillset	Mobile Development, Typescript
Estimated Time:	4 hours

Task 3.4	PDF handling
Scope:	Use library to export incident data to formatted PDF document.
Skillset	Typescript
Estimated Time:	10 hours

Task 3.5	Unit Testing
Scope:	Create mock data and test data flow, Test invalid/corrupted data.
Skillset	Unit Testing, Typescript
Estimated Time:	10 hours

CONNECTIVITY MODULE

Task 4.1	Protobuf Schema
Scope:	Setup protobuf based on provided interface
Skillset	Protobuf, Mobile development
Estimated Time:	6 hours

Task 4.2	Bluetooth Pairing Integration
Scope:	Setup Bluetooth connectivity and pairing logic
Skillset	Bluetooth, Network Communications, Mobile Development
Estimated Time:	12 hours

Task 4.3	Message Handling
Scope:	Setup events for when data is received from the connected device.
Skillset	Bluetooth, Protobuf, Design Patterns, Mobile Development
Estimated Time:	12 hours

Task 4.4	Database Access Object (DAO)
Scope:	Create a DAO to abstract away complex query logic and simplify database access.
Skillset	Databases, Typescript
Estimated Time:	10 hours

Task 4.5	Connectivity Testing
Scope:	Unit test data in/out. Unit test database access.
Skillset	Databases, Bluetooth, Networks, Unit Testing, Software Engineering principles
Estimated Time:	16 hours

APP SETTINGS MODULE

Task 5.1	Settings and Configuration Management
Scope:	Set up a Typescript equivalent of configurations from Java or add data to database for saved settings.
Skillset	Typescript, Data Handling
Estimated Time:	6 hours

Task 5.2	Startup Data Handling
Scope:	Ensure settings are loaded on app startup.
Skillset	Typescript, Design Patterns
Estimated Time:	2 hours

Task 5.3	Connect Screen to Data
Scope:	Implement Settings page by connecting it to data
Skillset	Databases, File handling, Typescript, UI Callbacks
Estimated Time:	6 hours

MODULE INTEROPERABILITY

Task 6.1	Connect Screens to Data
Scope:	Connect data to screens for all UI elements
Skillset	Databases, File handling, Typescript, UI Callbacks
Estimated Time:	6 hours per screen

Task 6.2	Unit Testing Connectivity
Scope:	Ensure screens and data interact as they should. Test all data flow.
Skillset	Typescript, Unit Testing, Software Engineering principles
Estimated Time:	14 hours

Task 6.3	Documentation
Scope:	Document all classes. Write README.md file.
Skillset	Software Engineering principles
Estimated Time:	3 hours per class. 10 hours for README.md

SCHEDULE / TIMELINE:

Our schedule will revolve around 2-week sprints giving us 12 sprints between project start and the end of the fall semester. Provided below are our projected milestones. As with any Agile development, code will always be functional on main with development occurring on user story branches and merging with a development branch.

Sprint 1	March 4 – March 17
Focus:	Project setup
Tasks:	1.1, 1.2, 1.4, 1.5
Deliverable:	Hello World app that can build and run with expected dependencies.

Sprint 2	March 18 – April 1
Focus:	UI and phone integration
Tasks:	1.3, 2.1, 2.3, 3.2
Deliverable:	App that allows navigation among unfinished screens with mock data

Sprint 3	April 2 – April 15
Focus:	Setup basic data handling
Tasks:	2.2, 3.2, 3.3
Deliverable:	App that can attach GPS/time to mock data and edit on simple minimum viable product (MVP) screen

Sprint 4	April 16 – April 30
Focus:	Network connections
Tasks:	3.1, 4.1, 4.2, 4.3
Deliverable:	App that can establish a pair connection and handle mock data.

Sprint 5	Fall Week 2 & 3
----------	-----------------

Focus:	Finalize data flow
Tasks:	3.1, 4.3, 4.4
Deliverable:	App that can accept mock data, edit data, and save data to the database.

Sprint 6	Fall Week 4 & 5
Focus:	Add Data to screens
Tasks:	6.1
Deliverable:	App that can view/use mock data on all screens. More polished than Sprint 5.

Sprint 7	Fall Week 6 & 7
Focus:	Settings management
Tasks:	5.1, 5.2, 5.3
Deliverable:	App with functional settings screen

Sprint 8	Fall Week 8 & 9
Focus:	Look and Feel + PDF export
Tasks:	3.4
Deliverable:	App with more polished and responsive UI that also can export incidents to PDF.

Sprint 9	Fall Week 10 & 11
Focus:	Unit testing
Tasks:	3.5, 4.5
Deliverable:	App with 70%+ code coverage for unit tests.

Sprint 10	Fall Week 12 & 13
Focus:	Testing & polish
Tasks:	4.5, 6.2
Deliverable:	App with strong inter-module testing and network failure handling.

Sprint 11	Fall Week 14 & 15
Focus:	UI testing and start formal documentation
Tasks:	2.4, 2.5

Deliverable:	App with minimal bugs and polished responsive UI with some formal documentation.
--------------	--

Sprint 12	Fall Week 16
Focus:	Documentation and final polish
Tasks:	6.3
Deliverable:	App with full formal documentation and no identified unaddressed bugs.

REQUIRED HARDWARE:

We need the IBDC handlebar device and a smartphone with the IBDC app. The device must sense close passes, flash a light, and take photos. The phone connects to the device, adds GPS and time, and lets the user view and export incidents.

THIRD PARTY CONTENT:

Third-party content for this project includes Protocol Buffers (Protobuf) for device-to-smartphone data exchange using a provided schema, ensuring efficient data serialization. The application will use an embedded SQLite compatible local database to hold incidents, images, and settings on the phone. For license plate capture, the app will integrate an Optical Character Recognition (OCR) library to extract plate text from selected incident photos. Finally, a Portable Document Format (PDF) generation library will be used to produce a formatted incident report output that includes incident metadata, notes, and the user-selected image.

QUALITY:

Quality for the Intelligent Bicycle Dash Cam (IBDC) application will focus on two primary areas: functionality and reliability. These areas directly support the goal of providing cyclists with an accurate and dependable tool for documenting unsafe passing incidents.

Functionality requires that the application fully implement all Key Requirements and Deliverables described in this document. Each requirement will be mapped to one or more test cases in a required traceability matrix to ensure complete coverage. All core features including Bluetooth pairing, incident import, data storage, incident review and editing, image selection, and report export must function as specified. Prior to final delivery, all high-priority acceptance tests must be passed, and no critical defects may remain open. Sponsor feedback at key milestones will confirm that implemented features meet both the written requirements and the intended user experience.

Reliability ensures that incident data is transferred, stored, and retrieved without corruption or unintended loss. All Protobuf messages received from the IBDC device will be validated before being saved to the database. The system must not exhibit data loss or corruption during normal operations such as importing, editing, deleting, and exporting incidents. The application must handle Bluetooth interruptions or malformed data gracefully, without crashing or compromising already stored information. During integration testing, the application must not crash under normal usage and demonstrate fluid performance on both android and iOS platforms.

Testing will occur at the unit, integration, and system levels. Unit tests will validate data parsing and database operations. Integration testing will verify correct data flow between Bluetooth communication, validation logic, and storage. System testing will validate complete user workflows from pairing through report export.

REFERENCES/SOURCES OF INFORMATION:

React Native

A framework for developing mobile applications cross platform between both iOS and Android using JavaScript or TypeScript and React instead of writing developing separate applications natively (iOS using Swift and Xcode/ Android using Kotlin and Android Studio). It is important to have a solid understanding of this framework as this is where the bulk of the development will happen. The [React Native Getting Started Guide](#) includes pertinent sections such as “The Basics”, “Environment setup”, “Workflow”, “UI and Interaction”, “Debugging”, “Testing”, and “Android and iOS guides”. The [React Native Tutorial](#) give a simple but helpful “Hello World” example that helps explains core ideas of JSX, components, state, and props. This example also shows how Native React differs from web React.

Expo

SDK tool chain (*referenced by React Native getting started guide) that makes developing with React Native easier. With Expo developers can generate product templates with preset configurations to fast-track development. In a stack relationship, Expo sits between the code (TypeScript) and the React Native Framework. In general, Expo preconfigures a React Native environment, has easy CLI tools, Built-in APIs, allows easier building and testing, and has less native setup, and emulation abilities. Note: When referencing these materials, we will be using Expo development build NOT Expo go. Expo Go is a generic application that can only run the native modules it is bundled with. To create Bluetooth, SQLite extras, and Protobuf helpers we will be able to develop native modules to work with the hardware; this can only be accomplished using Expo Developer. [Expo in 100 Seconds](#) is a quick overview explaining value and functionality of this tool set. The [Expo Getting Started Guide](#) provides relevant sections include “Introduction”, “creating a project”, “setting up your environment”, and “start developing”.

SQLite

a self-contained (embedded) relational database that lives in a single file on device. Every iOS and Android device already include SQLite support. SQLite is important to understand in storing and retrieving data on device is a requirement of this project. The [Expo SQLite Reference](#) provides a resource to the configuration settings, usage, and security materials for the library that is provided to use SQLite with the Expo developmental tool chain. In addition, the [SQLite Database in React Native and Expo Walkthrough](#) explains in how to implement an SQLite database in a React Native app. It covers setting up SQLite with Expo SDK, creating tables and CRUD operations, reading data from the SQLite database, handling offline data storage, and setting up a refresh mechanism.

Protobuf

A language-neutral, platform-neutral, extensible way of serializing structured data for use in communications protocols, data storage, and more. As per the requirements, a solid understanding of this protocol is crucial to communication with the hardware and software.

The [Protobuf Implementation and Documentation](#) is a GitHub repository with a pure JavaScript implementation with TypeScript support for Node.js. Relevant sections include installation, usage, and Examples (in particular, usage with TypeScript). This [Protobuf Overview](#) also provides general information about Protocol Buffers which includes programming guides, limitations, and techniques.

TypeScript

A programming language that sits on top of JavaScript and adds static typing and better tooling. TypeScript compiles into plain JavaScript that runs anywhere JavaScript runs. Working with React Native, it is important to understand this language. [The TypeScript Handbook](#) covers the basics, object types, type manipulation, classes and Modules. Additionally, the [React TypeScript Cheat sheet](#) covers the most common examples including Prop Types, Functional Components, Hooks, Class Components, Type defaultProps, Forms and Events, etc.

SCALABILITY

Scalability is not the primary design consideration for this project. The sponsor has no anticipated need for future expansion in either user load, or distributed operations. The system should therefore be optimized for reliability and simplicity within the defined scope rather than horizontal or vertical scaling.

OTHER SPECIAL CONSIDERATIONS

Predefined Protocol Buffer Schema

The client has already defined the Protocol Buffer (Protobuf) message schema used by the hardware device. The mobile application must remain fully compatible with this existing schema. Data exchanged between the smartphone and the device will conform to the provided proto specification.

Potential Future Data Sharing

Although the current project scope stores all incident data locally on the smartphone and does not include any cloud connectivity, the client has indicated that a foundation may request access to collected data for in the future for analysis and reporting purposes. To avoid limiting future expansion, the system should be designed with clear separation between data storage, business logic, and external interfaces.

Automated License Plate Extraction

The client has indicated a possible future enhancement to automatically extract license plate numbers from the series of images collected from the external device. While this capability is out of scope for the current project, special considerations should be given when designing the relational database scheme and application structure to ensure future extensibility.

PROCESS

PROCESS DESCRIPTION AND JUSTIFICATION

We will use Agile so we can build the most important features first and improve them over time. We will start with a basic version that connects to the device and saves incidents, then adds features like settings, review, and export. Each sprint will include planning, building, testing, and sponsor feedback. This approach is best because it finds problems early, adapts to changes, and keeps a working product ready for demos and final delivery.

TOOLS

The team will implement a variety of tools to enhance productivity and promote consistency within the project timeline and product quality. For project management, our team will be using [Taiga](#), a software to track the overall and scrum cycles goals. The SDK for this project will be [Expo](#). This is an Open-source platform for making universal native apps for Android, iOS, and the web with JavaScript/TypeScript and React/React Native. The versioning control software for this project will be [Git](#). Naturally, [GitHub](#) is where the team will manage, share, and collaborate on code. [Visual Studio](#) is integrated development environment to develop computer programs, websites, and mobile applications. This IDE works directly with Expo and has plugins for static analysis. [ESLint](#) will be the preferred static analysis tool to enforce coding standards.

ROLES AND RESPONSIBILITIES:

[Product Owner](#)

This role is responsible for defining the product vision and goals, managing and prioritizing the product backlog, accepting or rejecting completed work, and acts as the voice of the client.

[Scrum Master](#)

This role is responsible for ensuring Scrum is followed correctly, facilitating the ceremonies (sprint planning, Daily scrum “stand-up”, sprint reviews, and Retrospective), identifies and removes roadblocks, coaches team in Agile practices, and shields team from outside interference or scope changes.

[Developers](#)

This role is responsible for building the increment of working software each sprint, estimating effort, deciding how to implement the features, ensuring quality (testing, validation, integration), and self-organizing.

LOCATION OF PROJECT ARTIFACTS:

A GitHub Repository – [“Click Here”](#) – will be the location of all source code and artifacts for the project.

SPONSOR COMMUNICATIONS:

Communication with the client will occur on a regular cadence aligned with the conclusion of each sprint cycle. At the end of each cycle, the team will present the progress completed during the sprint and gather client feedback. The feedback will inform priorities, scope adjustments, and time commitments for the subsequent sprint.

Communication outside of the sprint review should take place via email, and any addition meeting will be scheduled at the client's discretion.

RISK MANAGEMENT

IDENTIFIED POTENTIAL RISKS:

Bluetooth Communication Failure

The application may fail to establish or maintain a stable Bluetooth connection with the IBDC device, resulting in interrupted or incomplete data exchange. This greatly impacts the reliability of incident data retrieval and is a moderate concern when considering Bluetooth stability across various mobile devices.

Data Integrity

Incident data, or images transmitted between the IBDC device and smartphone may become corrupted or lost during transfer. This loss compromises the accuracy and reliability of the incident records. The likelihood of this is moderate given wireless transmission is susceptible to interference or partial data delivery.

Cross-platform Functionality

Differences in Android and iOS development environments may result in inconsistent application performance and functionality across different platforms. These differences degrade experience and reliability of application. This can be a major concern if the applications are developed separately to each other instead of in tandem.

Scheduling Conflicts and Availability

Conflicting academic, work, or personal commitments may affect team availability during critical development phases. These conflicts may affect the overall quality and cohesion of the application and its integration phases, as well as increase complications if not handled correctly. This is a prevalent issue given the variability in semester workload and external obligations.

MITIGATION STRATEGIES:

Bluetooth Communication Failure

Communication failures can be mitigated through succinct retry logic and connection timeout handling. Beginning Bluetooth integration early in development would allow this refinement as well as testing across various devices to ensure stability.

Data Integrity

The impact of data loss can be mitigated by validating all incoming Protobuf messages before persisting data in the database, as well as implementing proper rejection and handling of malformed data. Furthermore, conducting integration testing using simulated interrupted transmissions would allow for early detection of potential edge cases.

Cross-Platform Functionality

The platform risks can be mitigated by consistent tracking and documentation of platform specific defects and disparities. This would allow both platform versions to maintain the same functionality when testing. In addition, these differences can be further minimized by building abstraction layers to handle differences in the platform frameworks.

Scheduling Conflicts and Availability

The conflicts and risks associated with team scheduling and availability can be mitigated through weekly progress meetings to monitor workload and task progression. Utilizing Git-based version control with structured branching and quality commit messages aids in combatting integration inconsistencies. Should it be required, an asynchronous workflow with steady documentation can also be adopted.